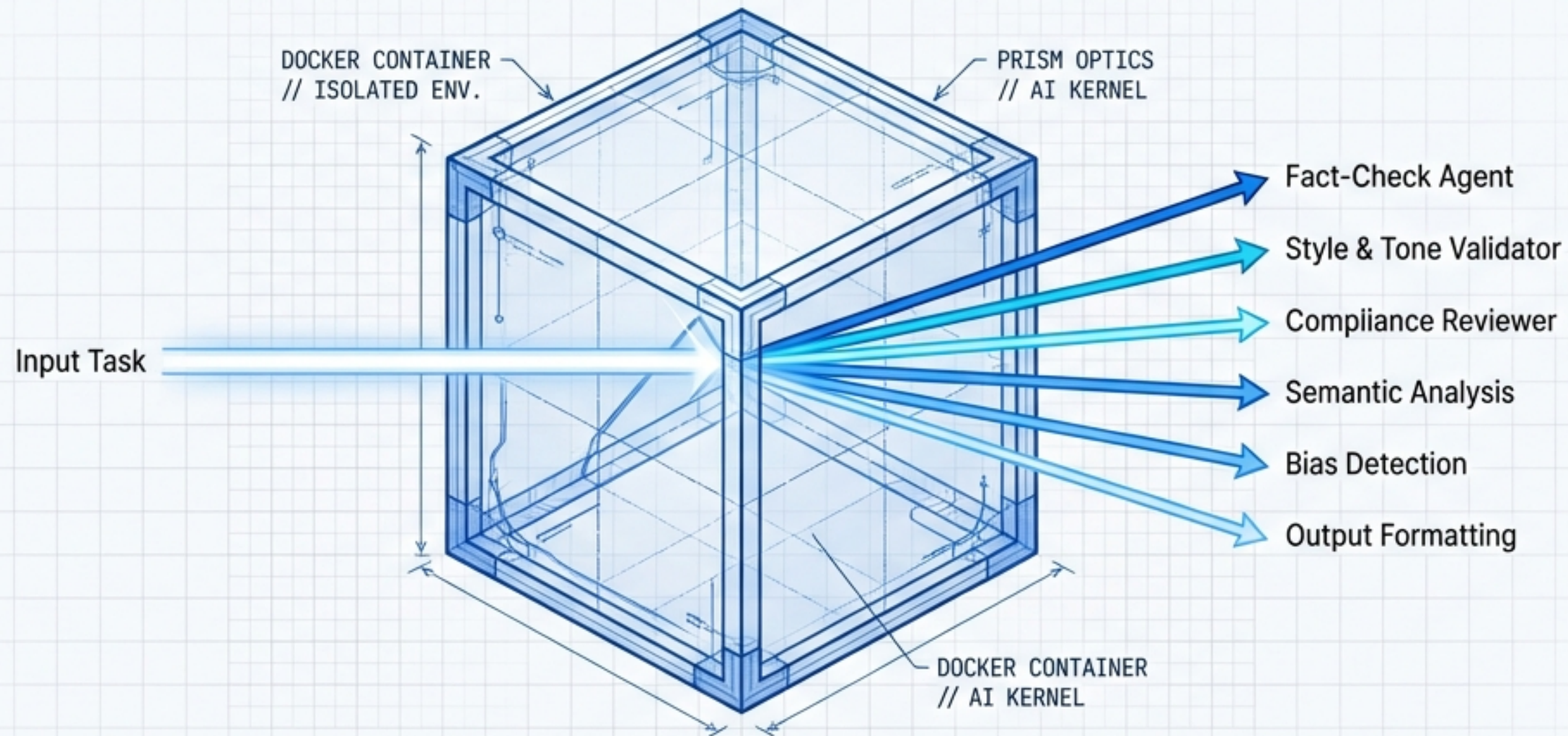


Multi-Agent Orchestration: The Claude-Flow Architecture

Implementing isolated, parallel AI workflows for enterprise-grade content validation.



Executive Summary

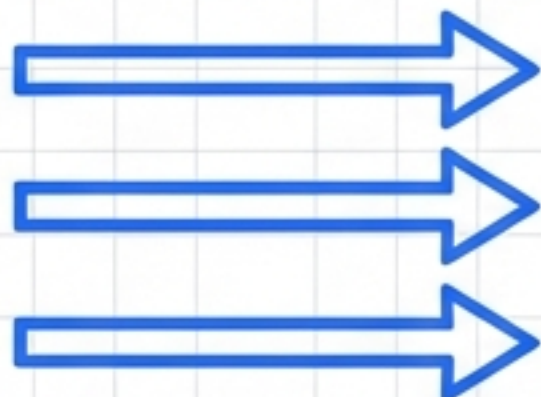
Bridging the Gap to Enterprise Orchestration

Standard LLM interfaces (Cework/Claude.ai) are insufficient for rigorous Quality Assurance. This document details a Dockerized solution engineered to solve specific scale and bias problems.



Isolation

True context separation to prevent hallucination and bias.



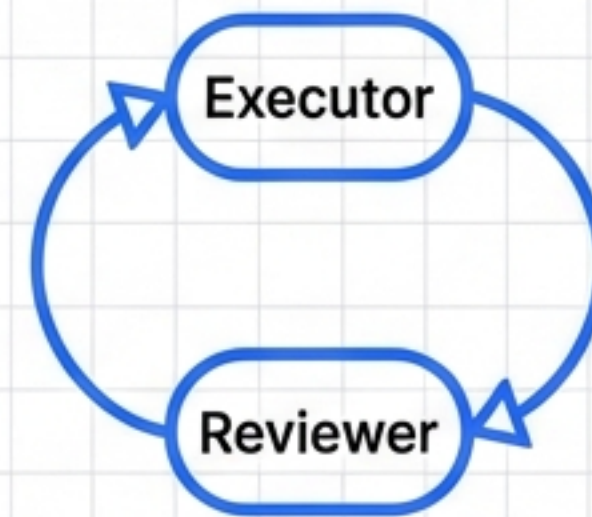
Parallelism

Agents running simultaneously rather than sequentially.



Persistence

Memory retention across sessions via separate volumes.



Workflow

A Two-Pass Executor/Reviewer pattern.

The Trigger: Content Validation at Scale

The Context:

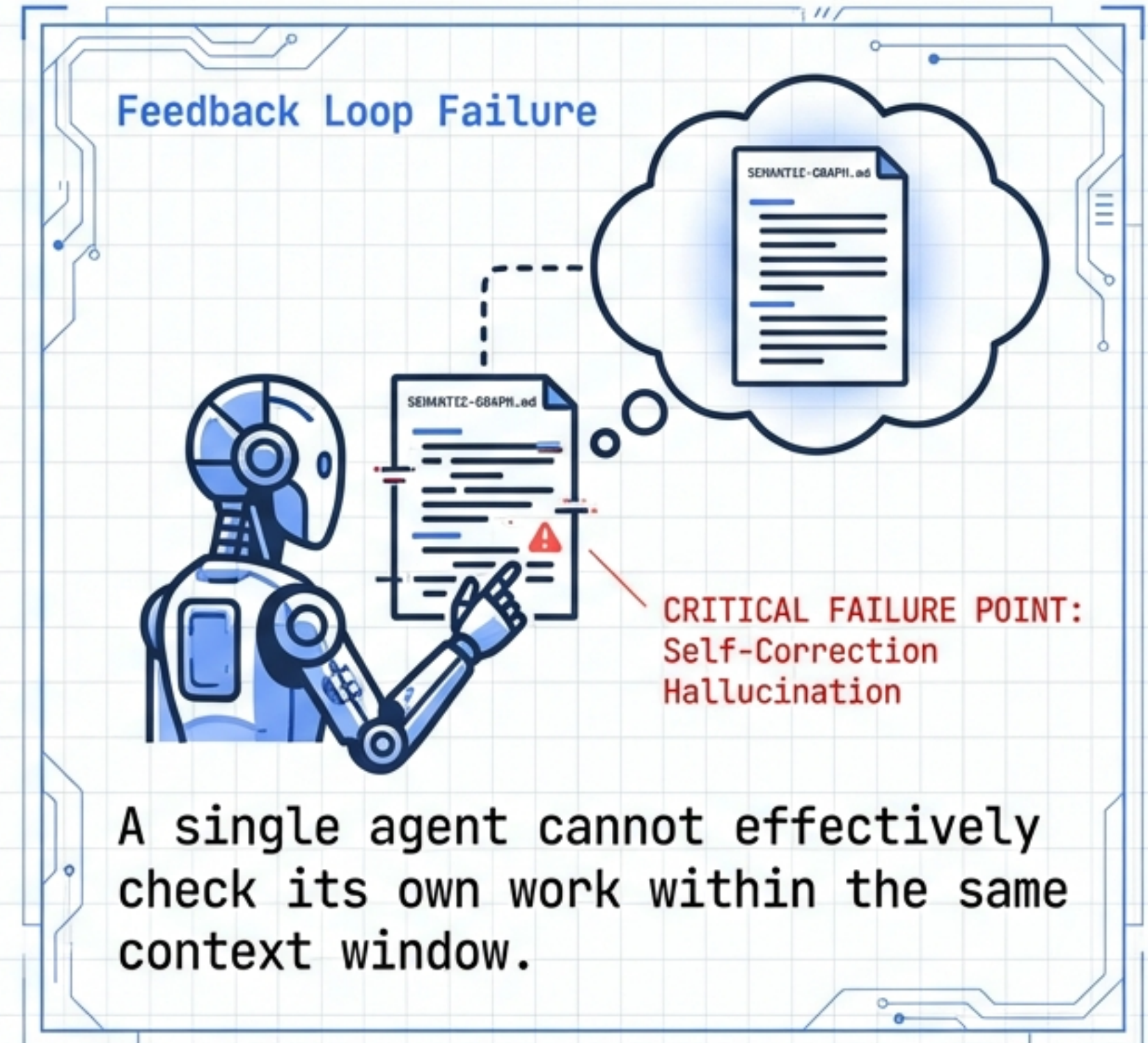
- **Scale:** 2.1GB repository size.
- **Complexity:** 271+ assets requiring management.

The Specific Task:

Generating and validating complex SEMANTIC-GRAPH.md files.

The Conflict:

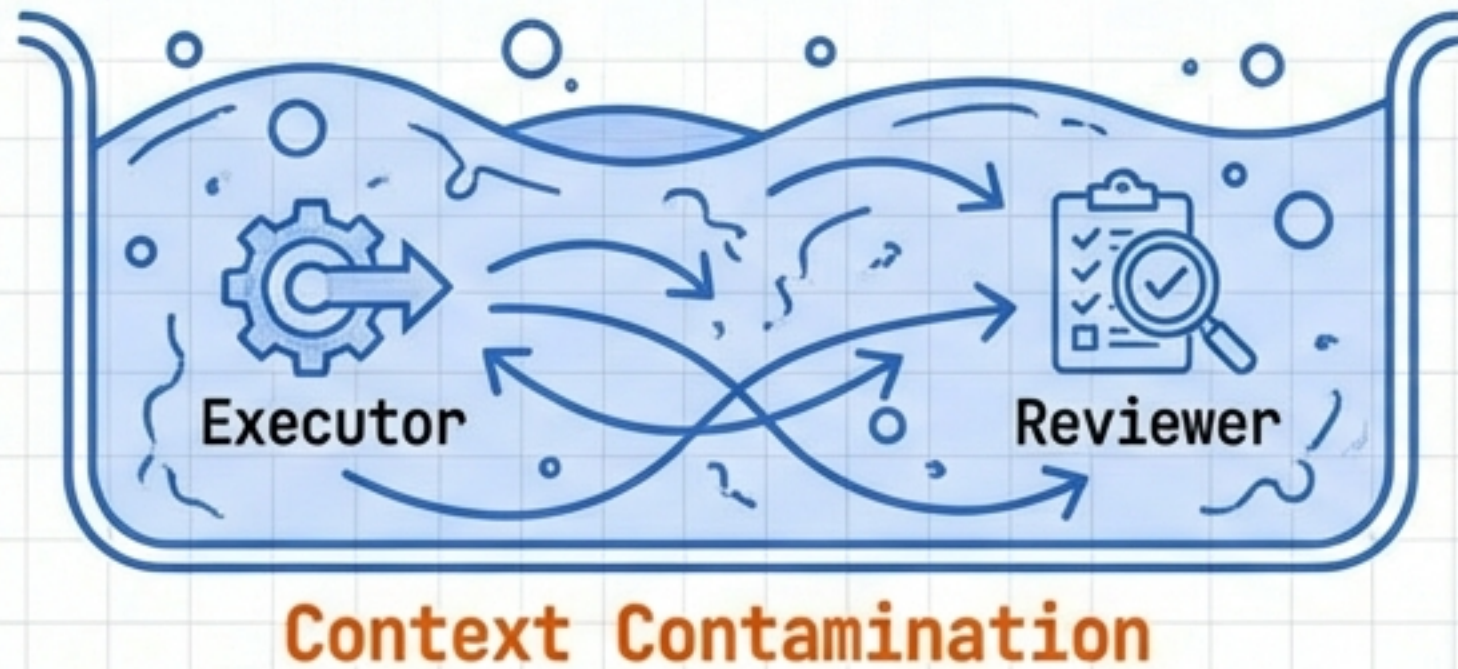
- High volume requires automation (Agentic creation).
- High standards require 'fresh eyes' validation.



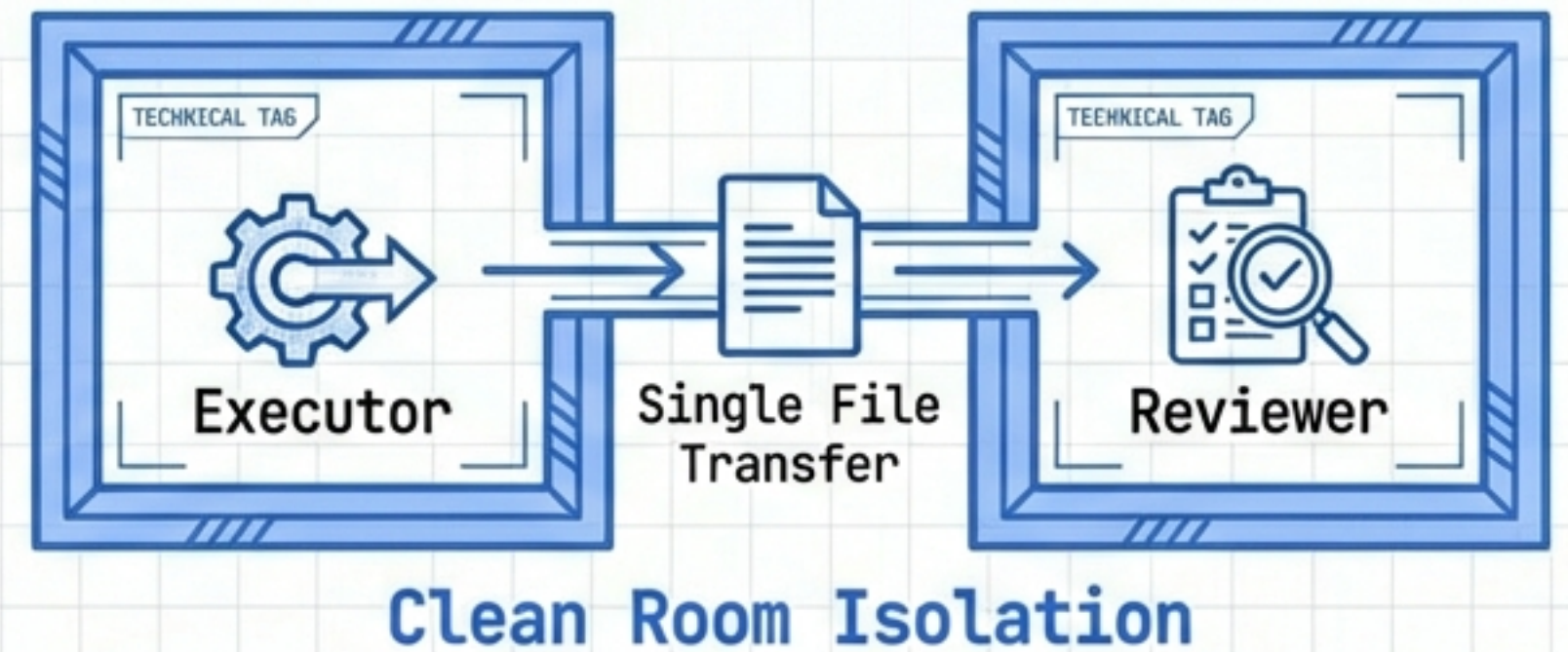
The Failure of Shared-Context Interfaces

Why 'Cowork' Interfaces Fail at QA

Shared Context (Standard Interface)

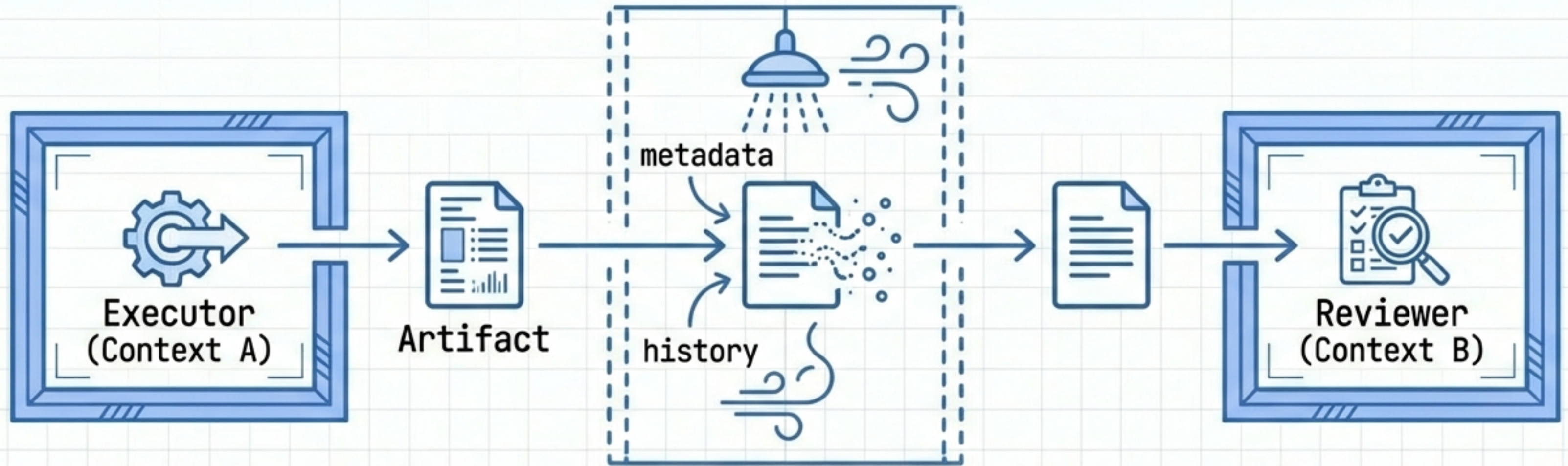


Docker Isolation (Claude-Flow)



Capability	Standard Interface	Claude-Flow
Executor Creation	✓ Capable	✓ Capable
Reviewer Validation	✓ Capable	✓ Capable
Separate Context	✗ Fail (Shared Bias)	✓ Success (Clean)
Parallel Execution	✗ Fail (Sequential)	✓ Success (Parallel)
Blind Review	✗ Fail (Biased)	✓ Success (Blind)

The Necessity of the 'Clean Room'



The Gap

In a shared context, a Reviewer agent is prejudiced by the Executor's 'thought process'.

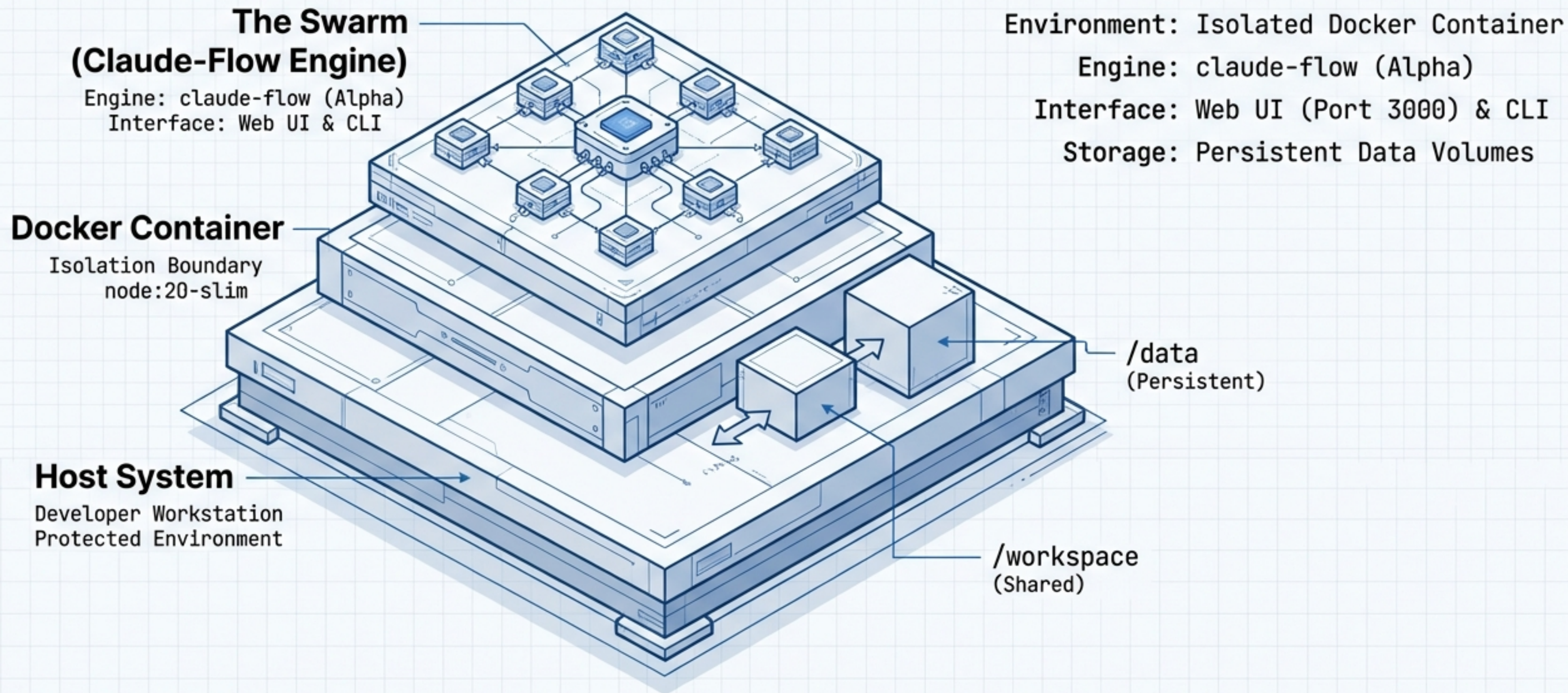
The Insight

To achieve genuine QA, the Reviewer must be BLIND to the creation process and interact only with the output.

The Takeaway

Isolation is not just a security feature; it is a correctness feature.

The Dockerized Claude-Flow Ecosystem



The Two-Pass Workflow Pattern

Decoupling creation from validation.

PASS 1: THE EXECUTOR



- **Input:** Raw data/requirements
- **Action:** Generates content
- **Output:** Saves to Artifact (e.g., SEMANTIC-GRAPH.md)



Artifact

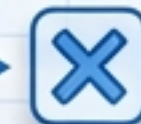
PASS 2: THE REVIEWER



- **State:** Spun up in a fresh container context
- **Input:** Reads the Artifact only (Blind)
- **Action:** Validates against rubric
- **Output:** Final Verdict / Rejection



VERDICT:
APPROVED

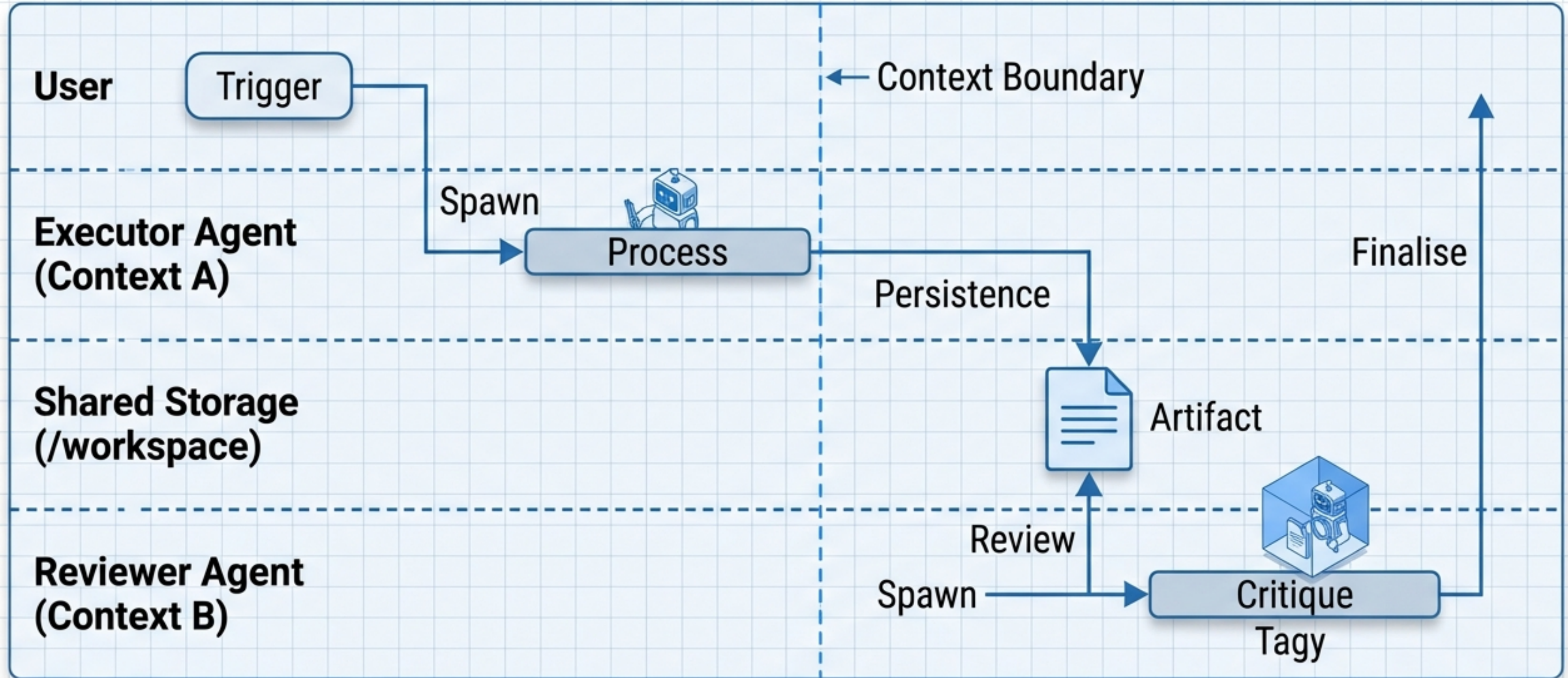


VERDICT:
REJECTED

OUTCOME: Unbiased, rigorous verification unclouded by the generation process.

Orchestrating the Swarm

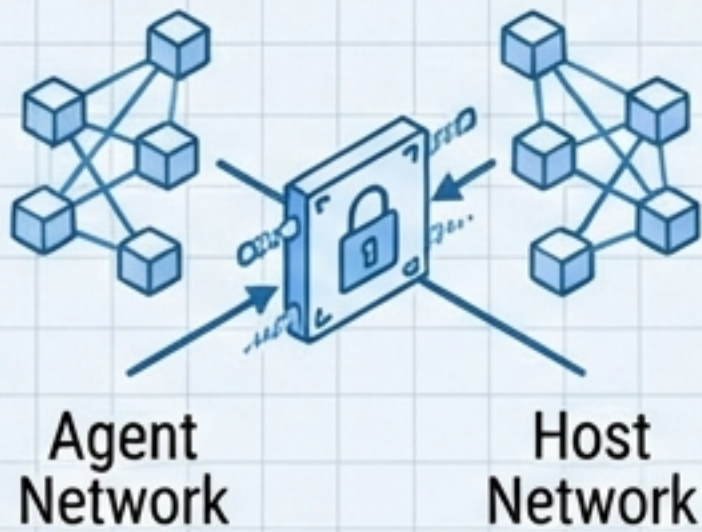
Execution Timeline & Data Flow



Defense in Depth

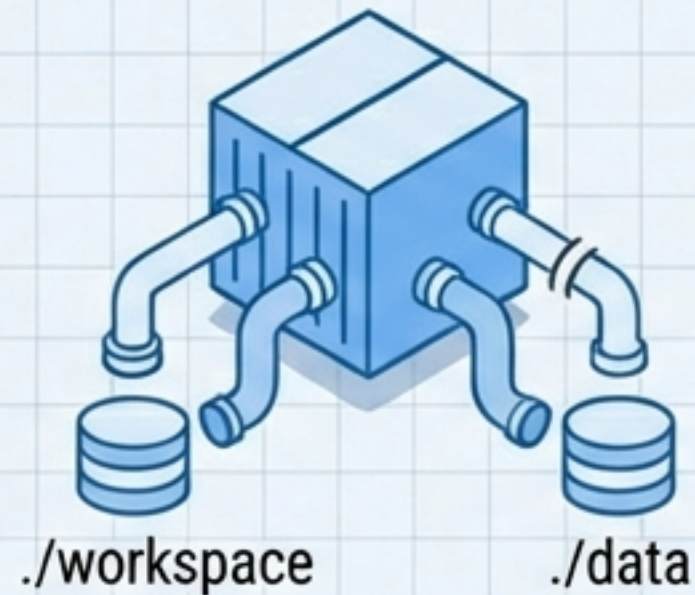
Security & Stability Architecture

Bridge Network



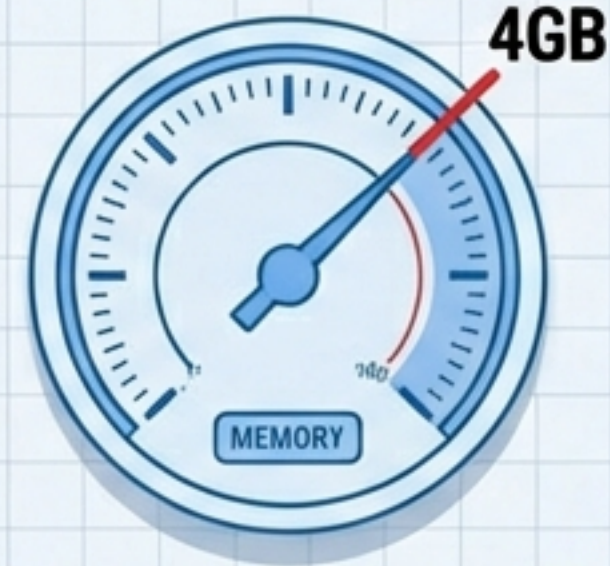
Controlled exposure.
Agents cannot
arbitrarily scan the
host network.

Volume Isolation



Agents only touch
explicit mounts:
./workspace and
./data.

Resource Caps



Strict 4GB memory
limit to prevent
runaway swarm
costs or crashes.

Host Protection



Host development
environment
remains untouched
by agent file ops.

Strategic Technology Choices

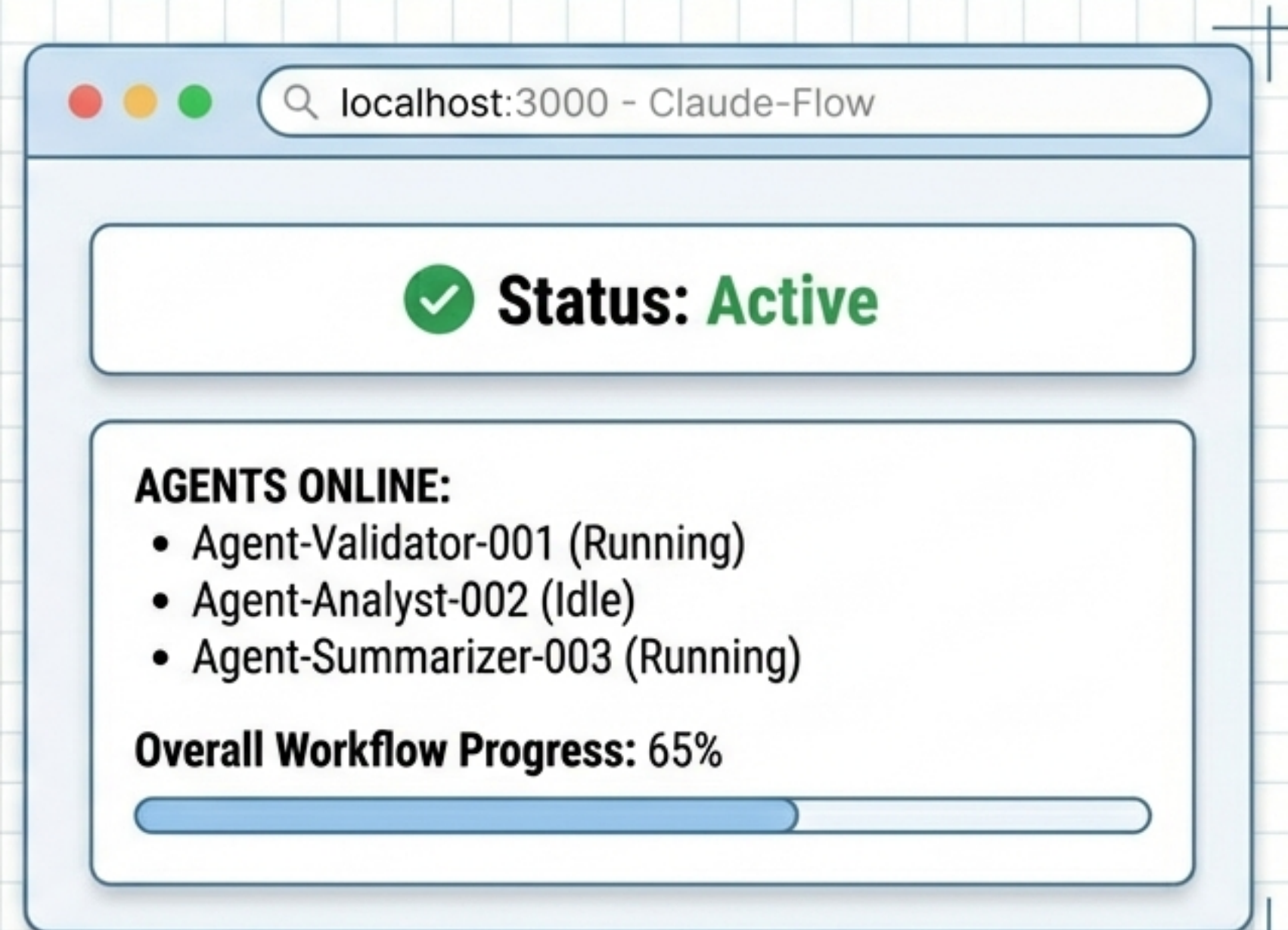
DECISION	RATIONALE
Base Image: node:20-slim	Minimal image size, LTS stability.
Orchestrator: claude-flow@alpha	Access to latest features including Web UI.
Tools: poppler-utils & imagemagick	Required for PDF processing and slide mosaics.
Memory Limit: 4GB	Sufficient for swarm ops, prevents runaway processes.
Storage: Separate Data Volume	Ensures persistence across container rebuilds.

Operationalising the Workflow

```
bash — 80x24

$ docker-compose up -d
[+] Running 3/3
X Network claude-flow_default Created
X Container claude-flow-1 Started

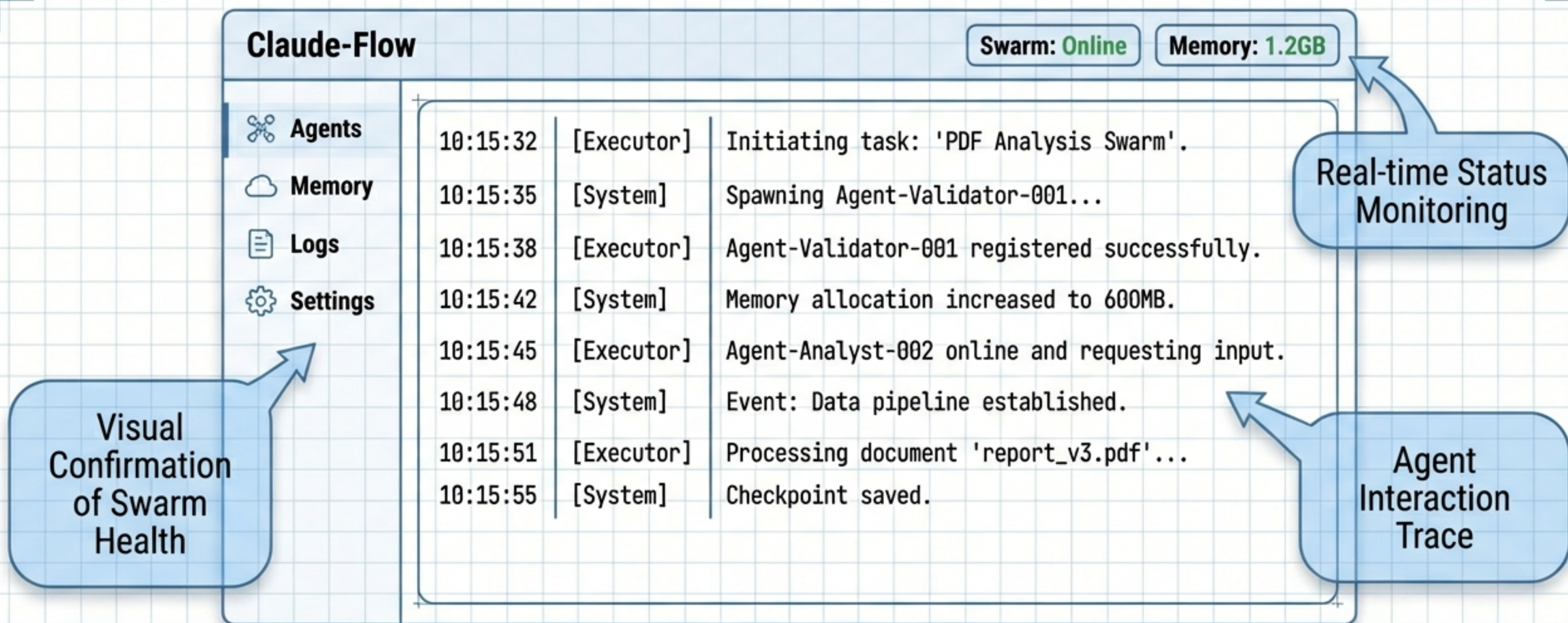
$ ./run-agent-swarm.sh --task='validate'
```



1. **Initialise:** One command startup via docker-compose.
2. **Interact:** Access Web UI via localhost.
3. **Execute:** Run helper scripts for agent spawning.
4. **Monitor:** Check logs via Docker or Web UI.

Monitoring and Interaction: The Web UI

A window into the black box.

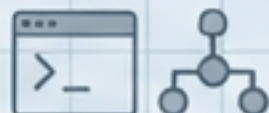


Must ensure '**claude-flow start-ui**' is running inside container.

Current Constraints & Workarounds

Transparency regarding Alpha status (v3.72.1)

CLI-based Control

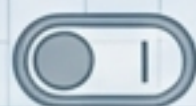


Constraint: Swarm control requires terminal.

Workaround: Use Web UI for monitoring only.



Manual Spawning

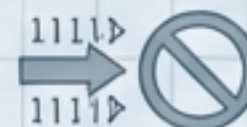


Constraint: Agents must be triggered manually.

Workaround: Use custom helper scripts.



No Real-Time Streaming



Constraint: Token streaming not yet implemented.

Workaround: Periodic polling.



Opaque Memory



Constraint: Memory not searchable in UI.

Workaround: Use CLI grep/search on volume.



A Production-Ready Quality Engine

We have moved beyond 'chatting' with AI to 'orchestrating' it.

01

True Parallelism

Efficiency at scale for large repositories.

02

Genuine Review

Unbiased quality control via 'Clean Room' architecture.

03

Enterprise Security

Isolated execution protects the host.

04

Scalability

Validated for the 2GB+ NotebookLM repository.

The Two-Pass pattern combined with Docker isolation provides the blueprint for trusted automated content generation.